

Sarah Benavides

CS 362 Lab 11

4/14/2026

Linux Forensics (Linux Auditing System)

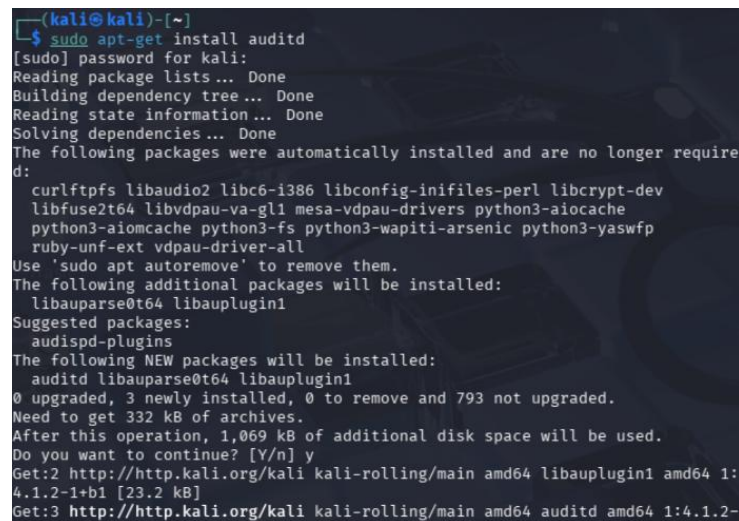
Part 1: Non-technical summary

This lab focuses on understanding how Linux systems track activity and assess security using built-in auditing tools. This lab started off by introducing the Linux auditing system, which helps monitor what is happening in a system by recording detailed logs of events such as user actions, system calls, and configuration changes. The lab walks through installing and enabling the auditing service, then explores how logs are generated and stored. It also demonstrates how to review and interpret this data using different tools that can search for specific events or generate summaries of system activity.

The second part of the lab introduces a security auditing tool that evaluates the overall health of the system. This tool scans for potential weaknesses, misconfigurations, and missing protections. After running a scan, it provides warnings and suggestions for improving security. The lab also shows how to investigate these issues, make small system changes, like updating network settings, and then re-running the scan to confirm improvements. Overall, this lab highlights how monitoring and regular security checks can help identify risks and strengthen a system's reliability.

Part 2: Technical summary

In this lab, we focused on Linux forensics, specifically the Linux auditing system. The Linux auditing system provides information about what is going on in the system in great detail. The first tasks we completed were installing, displaying the status, and then starting the daemon **auditd**.



```
(kali@kali)~$ sudo apt-get install auditd
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
 curlftpfs libaudio2 libc6-i386 libconfig-inifiles-perl libcrypt-dev
 libfuse2t64 libvdpau-va-gl1 mesa-vdpau-drivers python3-aiocache
 python3-aiomcache python3-fs python3-wapiti-arsenic python3-yaswfp
 ruby-unf-ext vdpau-driver-all
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
 libauparse0t64 libauplugin1
Suggested packages:
 audispd-plugins
The following NEW packages will be installed:
 auditd libauparse0t64 libauplugin1
0 upgraded, 3 newly installed, 0 to remove and 793 not upgraded.
Need to get 332 kB of archives.
After this operation, 1,069 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:2 http://http.kali.org/kali kali-rolling/main amd64 libauplugin1 amd64 1:
4.1.2-1+b1 [23.2 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 auditd amd64 1:4.1.2-
```

The output for installing **auditd** was very lengthy, therefore I only included the first half of it in the screenshot above.

```

(kali@kali)-[~]
$ systemctl status auditd | grep -i active
Active: inactive (dead)

(kali@kali)-[~]
$ systemctl start auditd

(kali@kali)-[~]
$ systemctl status auditd | grep -i active
Active: active (running) since Wed 2026-04-01 16:28:37 CDT; 15s ago

```

When we checked the status of **auditd**, it was originally inactive. In order to activate/start it, we used the command **systemctl start auditd**. After starting the daemon, we checked the status of it again and it then became active.

The audit daemon writes audit messages generated by the kernel **audit** interface. This auditing software is controlled by the file **/etc/audit/auditd.conf**. To list the content of the **/etc/audit/** folder we enter the command **sudo ls -l /etc/audit/**.

```

(kali@kali)-[~]
$ sudo ls -l /etc/audit/
total 24
-rw-r--r-- 1 root root 302 Dec  3 04:29 audisp-filter.conf
-rw-r--r-- 1 root root 901 Dec  3 04:29 auditd.conf
-rw-r--r-- 1 root root 107 Apr  1 16:25 audit.rules
-rw-r--r-- 1 root root 127 Dec  3 04:29 audit-stop.rules
drwxr-x-- 2 root root 4096 Apr  1 16:25 plugins.d
drwxr-x-- 2 root root 4096 Apr  1 16:25 rules.d

```

In this output, we see a breakdown of the files and directories related to the Linux auditing system. This list includes the type of permissions, the owner, the file size and time, and the file names. If we wanted to display the first lines of the Linux audit daemon's configuration file, we need to input the following command: **sudo cat /etc/audit/auditd.conf | head**.

```

(kali@kali)-[~]
$ sudo cat /etc/audit/auditd.conf | head
#
# This file controls the configuration of the audit daemon
#

local_events = yes
write_logs = yes
log_file = /var/log/audit/audit.log
log_group = adm
log_format = ENRICHED
flush = INCREMENTAL_ASYNC

```

This output lists the local events, write logs, log file, log group, log format, and the flush.

The next group of commands used are different variations for the audit daemon. We first used the command **sudo auditctl -s**. This command queries the status of the audit daemon.

```

(kali@kali)-[~]
$ sudo auditctl -s
enabled 1
failure 1
pid 34797
rate_limit 0
backlog_limit 8192
lost 0
backlog 0
backlog_wait_time 60000
backlog_wait_time_actual 0
loginuid_immutable 0 unlocked

```

This output displays a list showing the system is enabled, specifies how the kernel will handle critical errors, the process ID of **auditd**, the set limit in messages, the set maximum number of outstanding audit buffers allowed, the number of lost audit messages,

the number of current outstanding audit messages, the set time for the kernel to wait when the backlog limit is reached, and the actual time. In order to enable the auditing system and display one event for analysis, we entered the commands **sudo auditctl -e 1** and **sudo cat /var/log/audit/audit.log | grep -I syscall | head -n 1**.

```
(kali@kali)-[~]
$ sudo auditctl -e 1
enabled 1
failure 1
pid 34797
rate_limit 0
backlog_limit 8192
lost 0
backlog 3
backlog_wait_time 60000
backlog_wait_time_actual 0

(kali@kali)-[~]
$ sudo cat /var/log/audit/audit.log | grep -i syscall | head -n 1
type=SYSCALL msg=audit(1775078917.661:4): arch=c000003e syscall=44 success=yes
exit=60 a0=3 a1=7ffc5acd0520 a2=3c a3=0 items=0 ppid=34801 pid=34822 auid=4
294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none)
ses=4294967295 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(
null)ARCH=x86_64 SYSCALL=sendto AUDID="unset" UID="root" GID="root" EUID="root
" SUID="root" FSUID="root" EGID="root" SGID="root" FSGID="root"
```

A few important details in this display are the type, which is the type of event recorded, the message id, which is enclosed in the brackets, and the exe which is the path to the binary program.

If the user needed to create a custom and detailed report based on the audit log file content stored in **/var/log/audit.log** they would input the command **sudo aureport -if /var/log/audit/audit.log**.

```
(kali@kali)-[~]
$ sudo aureport -if /var/log/audit/audit.log

Summary Report
=====
Range of time in logs: 04/01/2026 16:28:37.403 - 04/01/2026 17:16:47.235
Selected time for report: 04/01/2026 16:28:37 - 04/01/2026 17:16:47.235
Number of changes in configuration: 4
Number of changes to accounts, groups, or roles: 0
Number of logins: 0
Number of failed logins: 0
Number of authentications: 0
Number of failed authentications: 0
Number of users: 3
Number of terminals: 6
Number of host names: 2
Number of executables: 4
Number of commands: 3
Number of files: 0
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 0
Number of anomaly events: 0
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 0
Number of process IDs: 26
Number of events: 109
```

This output displays a detailed summary report of the file **/var/log/audit/audit.log** and lists the numbers of users, logins, authentications, terminals, commands, and more. To list more functionalities of the **aureport** utility we enter the command **sudo aureport -au**.

```
(kali㉿kali)-[~]
$ sudo aureport -au

Authentication Report
=====
# date time acct host term exe success event
=====
<no events of interest were found>
```

Since I just installed the **auditd** daemon, there aren't any authentications yet. This is the same when checking the attempted logins using the command **sudo aureport -l**.

```
(kali㉿kali)-[~]
$ sudo aureport -l

Login Report
=====
# date time audit host term exe success event
=====
<no events of interest were found>
```

This output showed that no logins were found in this report.

In order to check the failed and successful events we use the commands **sudo aureport --failed** and **sudo aureport --success** respectively.

```
(kali㉿kali)-[~]
$ sudo aureport --failed

Failed Summary Report
=====
Range of time in logs: 04/01/2026 16:28:37.403 - 04/01/2026 17:34:27.538
Selected time for report: 04/01/2026 16:28:37 - 04/01/2026 17:34:27.538
Number of changes in configuration: 0
Number of changes to accounts, groups, or roles: 0
Number of logins: 0
Number of failed logins: 0
Number of authentications: 0
Number of failed authentications: 0
Number of users: 0
Number of terminals: 0
Number of host names: 0
Number of executables: 0
Number of commands: 0
Number of files: 0
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 0
Number of anomaly events: 0
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 0
Number of process IDs: 0
Number of events: 0
```

This failed summary report shows that nothing failed.

```

(kali@kali)-[~]
$ sudo aureport --success

Success Summary Report
=====
Range of time in logs: 04/01/2026 16:28:37.403 - 04/01/2026 17:35:00.638
Selected time for report: 04/01/2026 16:28:37 - 04/01/2026 17:35:00.638
Number of changes in configuration: 7
Number of changes to accounts, groups, or roles: 0
Number of logins: 0
Number of failed logins: 0
Number of authentications: 0
Number of failed authentications: 0
Number of users: 3
Number of terminals: 6
Number of host names: 2
Number of executables: 4
Number of commands: 3
Number of files: 0
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 0
Number of anomaly events: 0
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 0
Number of process IDs: 43
Number of events: 202

```

In this output, the success events included changes in configuration, number of users, number of terminals, number of host names, number of executables, number of commands, and number of process IDs. To look a bit deeper into the successful events in the system, we can also display start and end times for the success summary report using the command **sudo aureport -ts yesterday -te now --success**.

```

(kali@kali)-[~]
$ sudo aureport -ts yesterday -te now --success

Success Summary Report
=====
Range of time in logs: 12/31/1969 18:00:00.000 - 04/01/2026 17:48:48.164
Selected time for report: 03/31/2026 00:00:00 - 04/01/2026 17:48:48
Number of changes in configuration: 7
Number of changes to accounts, groups, or roles: 0
Number of logins: 0
Number of failed logins: 0
Number of authentications: 0
Number of failed authentications: 0
Number of users: 3
Number of terminals: 6
Number of host names: 2
Number of executables: 4
Number of commands: 3
Number of files: 0
Number of AVC's: 0
Number of MAC events: 0
Number of failed syscalls: 0
Number of anomaly events: 0
Number of responses to anomaly events: 0
Number of crypto events: 0
Number of integrity events: 0
Number of virt events: 0
Number of keys: 0
Number of process IDs: 50
Number of events: 233

```

Another way to retrieve more information from the auditing system, is using the command **sudo aureport -u --summary** to list a user report.


```
(kali@kali)-[~]
$ sudo aureport -u --summary

User Summary Report
=====
total  auid
=====
131  1000
60   0
48  -1
```

This output lists the total users and the au ID numbers. To list an event summary, we used the following command: **sudo aureport -e -i --summary**.

```
(kali@kali)-[~]
$ sudo aureport -e -i --summary

Event Summary Report
=====
total  type
=====
35  USER_ACCT
35  USER_START
35  CRED_REFR
34  CRED_DISP
34  USER_END
23  USER_CMD
12  LOGIN
12  CRED_ACQ
9   SERVICE_START
8   SERVICE_STOP
7   CONFIG_CHANGE
1   DAEMON_START
```

This event summary displays the total number of events as well as the type of event/username. To retrieve a more detailed output regarding the system's events, we can use **sudo aureport -e -ts yesterday -te now | head** to list the event report with times included.

```
(kali@kali)-[~]
$ sudo aureport -e -ts yesterday -te now | head

Event Report
=====
# date time event type auid success
=====
1. 04/01/2026 16:28:37 4472 DAEMON_START -1 yes
2. 04/01/2026 16:28:37 3 SERVICE_START -1 yes
3. 04/01/2026 16:28:37 4 CONFIG_CHANGE -1 yes
4. 04/01/2026 16:28:37 5 CONFIG_CHANGE -1 yes
5. 04/01/2026 16:28:37 6 CONFIG_CHANGE -1 yes
```

This report consists of dates, times, the event, type, the au ID and if it was successful.

To obtain a process report, we entered the command **sudo aureport -p | head**.

```
(kali㉿kali)-[~]
└─$ sudo aureport -p | head

Process ID Report
=====
# date time pid exe syscall audid event
=====
1. 04/01/2026 16:28:37 34797 ? 0 -1 4472
2. 04/01/2026 16:28:37 1 /usr/lib/systemd/systemd 0 -1 3
3. 04/01/2026 16:28:37 34822 /usr/sbin/auditctl 44 -1 4
4. 04/01/2026 16:28:37 34822 /usr/sbin/auditctl 44 -1 5
5. 04/01/2026 16:28:37 34822 /usr/sbin/auditctl 44 -1 6
```

This output lists the date, times, process ID, exe, syscall, au ID, and event for every process listed. The best way to access the system call events report is to enter the command **sudo aureport -s | head**.

```
(kali㉿kali)-[~]
└─$ sudo aureport -s | head

Syscall Report
=====
# date time syscall pid comm audid event
=====
1. 04/01/2026 16:28:37 44 34822 auditctl -1 4
2. 04/01/2026 16:28:37 44 34822 auditctl -1 5
3. 04/01/2026 16:28:37 44 34822 auditctl -1 6
4. 04/01/2026 16:35:01 1 37860 cron 0 11
5. 04/01/2026 16:39:01 1 39858 cron 0 24
```

This output lists the date, time, syscall, process ID, command, au ID and event of the system call.

In the last few steps of the first part of this lab, we experiment with the command **ausearch**. This utility is used to search the audit log file for specific events using a variety of keys. To begin, we used **id** to get the ID of the current logged in user.

```
(kali㉿kali)-[~]
└─$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),
25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(ne
tdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131
(kaboxer)
```

This output displays the user's identity as well as the groups on the Linux system. To dig a little deeper into the system's events, we can use the command **ausearch -ui 1000 | head** to search the events created by a specific user.

```
(kali㉿kali)-[~]
$ sudo ausearch -ui 1000 | head

time-->Wed Apr  1 16:38:12 2026
type=USER_ACCT msg=audit(1775079492.947:16): pid=39467 uid=1000 auid=1000 ses=6 subj=unconfined msg='op=PAM:accounting grantors=pam_permit acct="kali" exe="/usr/bin/sudo" hostname=kali addr=? terminal=/dev/pts/0 res=success'

time-->Wed Apr  1 16:38:12 2026
type=USER_CMD msg=audit(1775079492.947:17): pid=39467 uid=1000 auid=1000 ses=6 subj=unconfined msg='cwd="/home/kali" cmd=6C73202D6C202F6574632F61756469742F exe="/usr/bin/sudo" terminal=pts/0 res=success'

time-->Wed Apr  1 16:38:12 2026
type=CRED_REFR msg=audit(1775079492.955:18): pid=39467 uid=1000 auid=1000 ses=6 subj=unconfined msg='op=PAM:setcred grantors=pam_permit acct="root" exe="/usr/bin/sudo" hostname=kali addr=? terminal=/dev/pts/0 res=success'
```

The output we received goes in depth into each user event, listing specific information such as the type, msg, process ID, user ID, and more.

The last utility we focused on in this part was **autrace**. **Autrace** is used to trace particular processes. Before we used the **autrace** command, we need to delete the audit rules being used to avoid these rules clashing with the ones **autrace** adds. To do this, we use the command **sudo auditctl -D**.

```
(kali㉿kali)-[~]
$ sudo auditctl -D
No rules
```

After deleting the rules, we can now trace the binary less. We use the command **sudo autrace /usr/bin/less** to do this. After entering that command I received the output “sudo: autrace: command not found” even though I successfully installed **auditd**.

In the second part of this lab, we focused on security auditing using **Lynis**. **Lynis** is a security auditing tool for Linux and is used for compliance testing, penetration testing, vulnerability detection, and system hardening. To begin, I installed **Lynis** using the command **sudo apt-get install lynis**. To display the available commands **Lynis** provides, we input the following: **lynis show commands**.

```
(kali㉿kali)-[~]
$ lynis show commands

Commands:
lynis audit
lynis configure
lynis generate
lynis show
lynis update
lynis upload-only
```

After listing the available commands, we also want to analyze the settings of **Lynis**. We can display the settings using **lynis show settings**.


```
(kali@kali)-[~]
$ lynis show settings
# Colored screen output
colors=1

# Compressed uploads
compressed-uploads=0

# Use non-zero exit code if one or more warnings were found
error-on-warnings=0

# Language
language=en

# License key
license-key=[not configured]

# Logging of tests that have a different OS
log-tests-incorrect-os=1
```

```
# Upload server (ip or hostname)
upload-server=[not configured]

# Data upload after scanning
upload=no

# Verbose output
verbose=0

# Add --brief to hide descriptions, --configured-only to show configured items only, or --nocolors to remove colors
```

Since the output was fairly lengthy, I included the first and last few lines in the screenshot above.

When it comes time to perform a security audit of the system, we will use the command **sudo lynis audit system**. The information displayed in the output is stored in the log file **/var/log/lynis.log** and the data will be stored in **/var/log/lynis-report.dat**.

```
(kali@kali)-[~]
$ sudo lynis audit system

[ Lynis 3.1.6 ]

#####
###
Lynis comes with ABSOLUTELY NO WARRANTY. This is free software, and you are
welcome to redistribute it under the terms of the GNU General Public License.
See the LICENSE file for details about using this software.

2007-2025, CISOfy - https://cisofy.com/lynis/
Enterprise support available (compliance, plugins, interface and tools)
#####
###

[+] Initializing program
- Detecting OS ... [ DONE ]
- Checking profiles ... [ DONE ]
```

```

-[ Lynis 3.1.6 Results ]-

Warnings (1):
! Couldn't find 2 responsive nameservers [NETW-2705]
  https://cisofy.com/lynis/controls/NETW-2705/

Suggestions (47):
* Install libpam-tmpdir to set $TMP and $TMPDIR for PAM sessions [DEB-0280]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0280/

* Install apt-listbugs to display a list of critical bugs prior to each APT
  installation. [DEB-0810]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0810/

* Install apt-listchanges to display any significant changes prior to any u
  pgrade via APT. [DEB-0811]
  - Related resources
    * Website: https://cisofy.com/lynis/controls/DEB-0811/

```

This output was extremely long, so I only added the beginning portion along with the warnings and the start of the suggestions area. To dive a little deeper into any of the performed tests, I chose the one in the warning, we enter the command **sudo lynis show details NETW-2705**.

```

(kali@kali)-[~]
$ sudo lynis show details NETW-2705
[sudo] password for kali:
2026-04-03 16:51:49 Performing test ID NETW-2705 (Check availability two name
servers)
2026-04-03 16:51:49 Result: less than 2 responsive nameservers found
2026-04-03 16:51:49 Warning: Couldn't find 2 responsive nameservers [test:NET
W-2705] [details:-] [solution:-]
2026-04-03 16:51:49 Note: Non responsive nameservers can give problems for yo
ur system(s). Like the lack of recursive lookups, bad connectivity to update
servers etc.
2026-04-03 16:51:49 Suggestion: Check your resolv.conf file and fill in a bac
kup nameserver if possible [test:NETW-2705] [details:-] [solution:-]
2026-04-03 16:51:49 Hardening: assigned partial number of hardening points (1
of 2). Currently having 110 points (out of 169)
2026-04-03 16:51:49 ==

```

This detailed test report lists which test we are analyzing, the results, warnings, notes, suggestions, and hardening. After reading through this report, the suggestion area states to check your **resolv.conf** file. To check the content of **resolv.conf** we first need to switch from being a regular user to the superuser by entering command **sudo su**. Once we switched users, we then are able to display the content of the **etc/resolv.conf** file by using this command **cat /etc/resolv.conf**.

```

(kali@kali)-[~]
$ sudo su
(root@kali)-[/home/kali]
# cat /etc/resolv.conf
# Generated by NetworkManager
search cable.rcn.com
nameserver 192.168.0.1

```

In this output we can see the search domain and the DNS server this device is using. Since there is only one DNS server listed here, we have the ability to add more public DNS servers by using the commands **echo nameserver 8.8.8.8>> /etc/resolv.conf**, **echo nameserver 75.75.75.75>> /etc/resolv.conf**, and **echo nameserver 75.75.76.76>> /etc/resolv.conf**.

```
(root@kali)-[/home/kali]
# echo nameserver 8.8.8.8 >> /etc/resolv.conf

(root@kali)-[/home/kali]
# echo nameserver 75.75.75.75 >> /etc/resolv.conf

(root@kali)-[/home/kali]
# echo nameserver 75.75.76.76 >> /etc/resolv.conf
```

After adding the three additional public DNS servers, we want to go back and check the content of the file **/etc/resolv.conf** once more to confirm the servers were added.

```
(root@kali)-[/home/kali]
# cat /etc/resolv.conf
# Generated by NetworkManager
search cable.rcn.com
nameserver 192.168.0.1
nameserver 8.8.8.8
nameserver 8.8.8.8
nameserver 75.75.75.75
nameserver 75.75.76.76
```

When analyzing the output of the system security auditing, the warning states the system was unable to find more than one DNS server. Because we added three more DNS servers, we want to run the system security audit again to check if this warning will disappear.

```
-[ Lynis 3.1.6 Results ]-

Warnings (2):
! Nameserver 75.75.75.75 does not respond [NETW-2704]
https://cisofy.com/lynis/controls/NETW-2704/
! Nameserver 75.75.76.76 does not respond [NETW-2704]
https://cisofy.com/lynis/controls/NETW-2704/

Suggestions (48):
* Install libpam-tmpdir to set $TMP and $TMPDIR for PAM sessions [DEB-0280]
```

Those specific warnings technically did disappear, but unfortunately two different warnings appeared. These warnings state that the last two DNS servers that were added did not respond during the audit.

The last task we concentrated on in this lab was looking for the hardening score of the security system audit. This score is listed near the end of the audit and shows the hardening index, the number of tests performed and the plugins that were enabled.

```
Details:
Hardening index : 63 [#####]
Tests performed : 269
Plugins enabled : 1
```

Once fixing all the suggestions within the prior steps, the hardening index should improve.

Part 3: References

1. Lab 11: Linux Forensics (Linux Auditing System)
2. [Audit commands and their meanings](#)

3. Meaning of cat /etc/resolv.conf command